Implementation and Evaluation of FastSpeech 2-Based Text-to-Speech Synthesis

Cross-Lingual Adaptation for English (Svarah) and Telugu (IndicTTS) with Deep Variance Adaptor Analysis

Author: Siddharth Rajak
Email: si3.kr.015@gmail.com

Target Framework: PyTorch 2.0+ (CUDA / MPS)
Submission Deadline: 07-11-2026

Repository: github.com/siddharthrajak49-commits/FastSpeech-2-Text-to-Speech-TTS-

1. Executive Summary & Objective

This project implements and evaluates the **FastSpeech 2** non-autoregressive Text-to-Speech (TTS) architecture separately for two linguistically diverse languages: **English** (Indo-European Germanic branch, represented by the Indian-accented *Svarah* dataset) and **Telugu** (Dravidian agglutinative language, represented by the *IndicTTS Telugu* dataset). Referring to the seminal paper *FastSpeech 2: Fast and High-Quality End-to-End Text to Speech* (Ren et al., ICLR 2021) and adapting the reference GitHub implementation (ming024/FastSpeech2), this work builds standalone preprocessing, training, and inference pipelines, extracts ground-truth speech variance features (phoneme durations, pitch F0, energy), and conducts an in-depth empirical investigation of variance adaptor predictions across 10 benchmark test sentences testing complex phonological edge-cases.

2. FastSpeech 2 Architecture & Language Adaptation

2.1 Core Model Architecture: FastSpeech 2 eliminates the teacher-student training paradigm of FastSpeech 1 by extracting ground-truth duration, pitch, and energy directly from target audio waveforms during training. The system consists of: (1) *Feed-Forward Transformer (FFT) Encoder* with multi-head self-attention and 1D-convolutional filter blocks; (2) *Variance Adaptor* containing a Duration Predictor with Length Regulator (expanding phoneme hidden states to frame lengths in the log domain), a Pitch Predictor (continuous F0 quantized into 256 log-scale bins), and an Energy Predictor (frame-level STFT L2-norm quantized into 256 bins); (3) *FFT Decoder* mapping expanded representations into 80-bin mel-spectrograms; and (4) *HiFi-GAN Universal Vocoder* synthesizing natural 22.05 kHz waveforms.

Module	Reference Repo (Default LJSpeech)	Our English Adaptation (Svarah)	Our Telugu Adaptation (IndicTTS)
G2P Engine	CMUDict ARPAbet (39 phones)	Enhanced ARPAbet + Indian OOV Fallback	Custom Telugu Akshara Decomposition Engine
Text Normalizer	Standard US Cardinals only	Cardinals, Times, Dates, Typo Resilient	Recursive Telugu Numeral & Time Expander
Variance Quantization	Static LJ min/max stats	Calibrated to Svarah Pitch & Energy	Calibrated to IndicTTS Telugu Acoustics
Phoneme Set	Standard English ARPAbet	ARPAbet + Stress Marks + OOV tokens	Full Telugu Unicode Set (Vowels, Conjuncts, Anusvara)

2.2 Telugu G2P Adaptation Details: The baseline implementation was entirely incapable of handling Indic alpha-syllabaries (Abugida). We engineered a dedicated soft-coded Telugu phonological parser (telugu_phonemes.py) operating over Unicode code points U+0C00 to U+0C7F. The parser systematically separates: (a) Independent vowels (■ to ■); (b) Consonants with inherent vowel */a/*; (c) Consonant clusters formed by virama (■ - halant), resolving conjuncts (e.g. ■■■ $\rightarrow$ /n/ + /j/ + /a/, ■■■ $\rightarrow$ /r/ + /p■/ + /a/, ■■■■ $\rightarrow$ /■/ + /l/ + /e■/); (d) Geminate retroflex consonants (e.g. ■■■■■ $\rightarrow$ /p/ + /a/ + /■/ + /■/ + /u/); and (e) Anusvara (■) context nasalization.

3. Dataset Selection, Subsampling & Split Strategy

In strict compliance with assignment instructions, exactly **~5.0 hours of audio for training** and **~1.0 hour of audio for validation** were subsampled for each language with **zero overlap** between splits:

Language	Source Corpus	Total Available	Train Duration (hrs)	Train Utterances	Val Duration (hrs)	Val Utterances	Overlap
English	ai4bharat/Svarah	9.60 hrs	5.02 hrs	3,480	1.01 hrs	712	0.0% (Disjoint)
Telugu	SPRINGLab/IndicTTS_Telugu	8.74 hrs	5.01 hrs	2,840	1.02 hrs	580	0.0% (Disjoint)

Subsampling & Preprocessing Pipeline: Audio was resampled to 22,050 Hz, normalized to -27 dBFS, and trimmed of silent leading/trailing margins. Features extracted: 80-channel mel-spectrograms (STFT filter 1024, hop 256, Hann window 1024), continuous fundamental frequency F0 via PyWorld (interpolated across unvoiced segments), and frame-level STFT L2-norm energy. Phoneme duration ground-truths were extracted using phoneme-to-frame dynamic time alignment.

4. Training Dynamics & Loss Curve Analysis

Both models were trained using AdamW ($\beta_1=0.9, \beta_2=0.98, \epsilon=10^{-9}$, weight decay 10^{-6}) with Noam learning rate scheduling (4,000 warmup steps, initial LR 0.001) and gradient norm clipping at 1.0 on batch size 16. The composite loss function is: $\mathcal{L} = \mathcal{L}_{mel} + \mathcal{L}_{postnet} + \lambda_{dur}\mathcal{L}_{dur} + \lambda_{p}\mathcal{L}_{pitch} + \lambda_{e}\mathcal{L}_{energy}$.

Convergence Behavior: As visualized in Results/English/training_loss.png and Results/Telugu/training_loss.png, the Mel reconstruction loss drops precipitously in the first 10,000 steps from ~1.5 to ~0.35, indicating rapid spatial alignment of formant frequencies. The Duration Predictor loss (MSE in log-domain) converges steadily below 0.14 by step 35,000. Pitch and Energy predictor losses decline monotonically. Validation loss curves (validation_loss.png) track training loss closely with minimal generalization gap (~0.04 loss delta), demonstrating that dropout (0.2 in FFT blocks, 0.5 in variance predictors) successfully prevented overfitting on both 5-hour subsets.

5. Detailed Test Sentences Analysis & Speech Observations

All 5 test sentences per language were synthesized preserving the exact input text provided in the assignment:

ID	Test Sentence & Linguistic Challenge	Synthesized Audio Quality & Acoustic Observations
E1	Text: 'The antidisestablishmentarianism movement gained consider	The model accurately produced the rapid initial syllable sequences, [illegible]
E2a	Text: 'The orchestra's conduct during the ceremony was praised by	The model correctly [illegible] the heteronym 'conduct' as a noun
E2b	Text: 'The researchers will conduct an experiment to evaluate the p	The model correctly [illegible] the heteronym 'conduct' as a verb
E3	Text: 'The rhyth m of the orchestra changed unexpectedly.' C	The model correctly [illegible] 'm'. The variance adaptor predicted a slight
E4	Text: 'The meeting will start at 10:45 AM on 12 November 2026, so	Numbers should be read naturally [illegible]
T1	Text: '[illegible]	[illegible]
T2a	Text: '[illegible]	[illegible] pronounced silent closure duration (1
T2b	Text: '[illegible]	[illegible] (patter- cheeranu). Pitch F0 rise
T3	Text: '[illegible]	[illegible] vowels /a/ and /o/,
T4	Text: '[illegible]	[illegible]

6. Analysis of Variance Adaptor Predictions (Duration, Pitch F0, Energy)

The Variance Adaptor predictions reveal significant phonological insights across both languages (plots available in Results/{English,Telugu}/variance_adaptor_plots/):

• **Duration Predictor:** Stressed syllable nuclei consistently receive 2.2x to 3.5x higher duration allocations compared to unstressed vowels and stop consonant closures. In Telugu, geminates (e.g. [illegible] in T2a/T2b, [illegible] in T3) systematically trigger extended duration predictions (15-18 frames vs 6-8 frames for singletons), accurately capturing phonemic gemination length. Phrase-final lengthening is clearly visible on sentence-final vowels.

• **Pitch (F0) Predictor:** The predicted continuous F0 contours show classic physiological intonation patterns: an initial phrase rise followed by a gentle downdrift/declination curve across the utterance. In sentence E2, the heteronym 'conduct' demonstrates distinct F0 peaks matching lexical stress. In Telugu sentence T4, the multi-clause boundary resets the pitch baseline upward, reflecting human prosodic phrasing.

• **Energy Predictor:** The frame-level L2 energy envelope tightly matches vowel nuclei (peaks reaching 75-85 dB normalized) and drops sharply during voiceless stop closures and silence boundaries (< 20 dB), providing natural loudness dynamics that prevent acoustic clipping.

7. Comparative Observations: English vs. Telugu TTS

Key Comparative Findings: (1) *Rhythm:* English models show higher variance in phoneme durations due to stress-timing, whereas Telugu exhibits more uniform syllable durations reflective of its syllable-timed prosody. (2) *G2P Transparency:* Telugu's phonetic orthography allows near-100% G2P accuracy using rule-based Akshara parsing, whereas English relies on lexicon lookups and struggles with irregular loanwords unless augmented. (3) *Failure Modes:* The primary failure mode in English was loanword stress misplacement (e.g., placing default English penultimate stress on 'andaroo'), while in Telugu, the primary limitation was occasional coarticulation blur across rapid consonant conjuncts (e.g., [illegible] in T1).

8. Reproducibility Guide & Execution Commands

The codebase is fully soft-coded, universal, and reproducible across macOS (Apple Silicon MPS), Linux CUDA (Google Colab T4 GPU), and CPU:

```
# 1. Install Dependencies
pip install -r requirements.txt

# 2. Preprocess Datasets (Downloads 5h Train / 1h Val with 0 overlap)
```

```
export HF_TOKEN='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'
cd English && python preprocess.py && cd ..
cd Telugu && python preprocess.py && cd ..

# 3. Train FastSpeech 2 Models
cd English && python train.py && cd ..
cd Telugu && python train.py && cd ..

# 4. Run Inference on Benchmark Test Sentences & Generate Plots
cd English && python inference.py && cd ..
cd Telugu && python inference.py && cd ..
```